Smart Parking : répartition du travail

Mise à jour : 2 novembre 2026 · Équipe : **Eya (A)**, **Amal (B)**, **Maissa (C)** Encadrant : Dr. Ing. Mohamed-Bécha Kaâniche

Ce document décrit **qui fait quoi** pendant les 10 sprints, **comment** nous travaillons ensemble (Git, revues, démos) et **à quoi on reconnaît qu'une tâche est terminée**.

---

1. Principe : un roulement entre trois domaines

Le projet a trois domaines techniques :

Domaine	Contenu	Dossier
IoT et IA	Caméra, Raspberry Pi, dataset, modèle YOLO, occupation, anomalies, client MQTT	`iot/`
Serveur	VM cloud, Mosquitto, WildFly (Jakarta EE), MongoDB, API REST, JWT, guidage, alertes	`api/`
PWA	Interface conducteur, interface exploitant, plan SVG, notifications push	`pwa/`

Chaque semaine compte **trois créneaux** :

Créneau	Durée	Nature
Lun–Mar	2 jours	Travail individuel, un domaine par personne
Mer–Jeu	2 jours	Travail individuel, un domaine par personne
Ven	1 jour	**Commun** : intégration, pull requests, démo, documentation

À chaque créneau individuel, les trois membres travaillent dans **trois domaines différents** (jamais deux personnes sur le même domaine), et le domaine change d'un créneau à l'autre. Ainsi :

- tout le monde connaît les trois parties du projet (pas de « spécialiste unique » qui bloque l'équipe) ;
- on évite les conflits Git : deux personnes ne modifient jamais les mêmes fichiers en même temps ;
- le bus factor est de 3, pas de 1.

Total sur les 10 sprints (hors tâches déjà réalisées le 2 novembre) :

Profil	Membre	IoT et IA	Serveur	PWA	Total
A	**Eya**	5	7	5	17 créneaux
B	**Amal**	6	6	5	17 créneaux
C	**Maissa**	6	7	4	17 créneaux

2. Comment nous travaillons ensemble

Notre méthode est l'**Extreme Programming (XP)** décrite dans le cahier des charges : itérations courtes, tests tôt, intégration continue, retours réguliers de l'encadrant.

2.1 Rythme d'un sprint (une semaine)

Moment	Ce qu'on fait
Lundi matin (15 min, tous)	Mini-planning : relire les livrables du sprint, vérifier que les contrats dont chacun dépend sont à jour, signaler les blocages.
Lun–Mar, Mer–Jeu	Chacun travaille dans son domaine, sur sa branche. Message court dans le groupe en début et fin de créneau (« je fais X », « fait : X, reste : Y, blocage : Z »).
Jeudi soir	Chacun ouvre sa pull request (même incomplète, en *draft*) pour que la revue du vendredi soit rapide.
Vendredi (journée commune)	1) revues et fusion des PR ; 2) test d'intégration sur la maquette ; 3) **démo du vendredi** ; 4) documentation ; 5) tag Git `sprint-N` ; 6) rétrospective (10 min : ce qui a marché, ce qu'on change).

2.2 Règles Git

Le dépôt est unique (`projet_smart_parking`), branche protégée par convention : **on ne pousse jamais directement sur `main`**.

1. **Une branche par personne et par créneau**, nommée `domaine/prénom-sprintN-créneau`, par exemple `iot/eya-s3-lunmar`, `api/amal-s3-lunmar`, `pwa/maissa-s3-lunmar`.
2. **Au moins un commit à chaque créneau** (chacune commite à chaque créneau). Commits petits, message en français à l'impératif : « Ajoute le calcul du recouvrement boîte/polygone ».
3. **Pull request vers `main`** à la fin du créneau. Elle décrit : ce qui est fait, comment le tester, ce qui reste.
4. **Revue obligatoire par une autre personne** avant fusion, selon un tour fixe : **Eya relit Amal, Amal relit Maissa, Maissa relit Eya**. Le relecteur lance le code ou le test décrit dans la PR, il ne lit pas seulement le diff.
5. **Fusion par le relecteur**, en « squash and merge » pour garder un historique lisible.
6. Avant de commencer un créneau : `git checkout main && git pull`, puis création de la branche. Si `main` a bougé pendant le créneau : `git pull --rebase origin main` avant la PR.
7. **Jamais de fichier volumineux ou secret dans Git** : pas de `.venv/`, de modèles `*.pt` (sauf `iot/models/`), de photos de test, de `.env`, de clés VAPID, de mots de passe ni de clés privées TLS. Le `.gitignore` les exclut déjà ; les secrets vont dans un `.env` local et dans les variables de la VM.
8. Chaque vendredi, après la démo : **tag `sprint-N`** sur `main`.

2.3 Propriété des dossiers et conflits

- Chaque dossier (`iot/`, `api/`, `pwa/`) a, à un instant donné, **une seule personne qui y écrit**. Le roulement garantit cela.
- Les fichiers partagés (`docs/contrats.md`, `README.md`, `places.exemple.json`) sont **modifiés uniquement le vendredi** ou par une PR dédiée relue par les deux autres. Un changement de contrat sans accord des trois est interdit.
- En cas de conflit Git : la personne qui a ouvert la PR le résout, le relecteur vérifie.

2.4 Les contrats : ce qui nous permet de travailler en parallèle

Comme chacun travaille sur un domaine différent, le travail d'une personne dépend souvent de celui d'une autre. Les **contrats** (dans `docs/contrats.md`) fixent les interfaces dès le début :

Contrat	Contenu	État
MQTT	Topics, format JSON de l'état des places, des événements d'anomalie et du statut du Pi	Rédigé, à valider ensemble (sprint 1)
Polygones	Format du fichier des places (`places.exemple.json`)	Rédigé, à valider ensemble (sprint 1)
API REST	Routes, formats de requête et de réponse, codes d'erreur, rôles requis	À rédiger (sprint 1, vendredi)
Push	Format des notifications et de l'abonnement Web Push	À rédiger (sprint 1, vendredi)
Plan de la maquette	Dimensions, numérotation des places, zones, graphe (nœuds et arêtes)	À rédiger (sprint 1, vendredi)

Tant que la partie réelle n'existe pas, on travaille avec des **données simulées** conformes au contrat : un script qui publie de faux messages MQTT, des réponses d'API fixes (« mock »), des fichiers JSON de test. Ainsi, le domaine A n'attend pas le domaine B.

2.5 Définition de « terminé »

Une tâche est terminée seulement si :

1. le code est sur `main` via une PR relue ;
2. il respecte le contrat concerné (ou le contrat a été mis à jour avec l'accord des trois) ;
3. il est **testé** : test unitaire quand c'est possible, sinon un scénario de test écrit dans la PR et exécuté par le relecteur ;
4. il se lance avec les commandes du README, sur une machine propre ;
5. la partie de documentation correspondante est à jour dans `docs/` .

2.6 Si on prend du retard

- Un blocage de plus d'**une demi-journée** est signalé immédiatement dans le groupe, sans attendre le lundi.
- Une tâche non terminée le jeudi soir est **terminée ensemble le vendredi** avant de commencer autre chose ; ce qui reste passe en tête du sprint suivant.
- L'ordre de priorité en cas de coupe : 1) la chaîne complète caméra → serveur → écran ; 2) la réservation et le guidage ; 3) les anomalies ; 4) les statistiques et les finitions.
- Les risques du cahier des charges (lumière, puissance du Pi, polygones mal configurés, coupure Internet) ont chacun une parade ; l'ordinateur portable de repli remplace le Pi si besoin.

2.7 Communication

- Un groupe de discussion pour les messages courts ; les décisions importantes sont notées dans `docs/decisions.md` (date, décision, raison).
- Les questions pour l'encadrant sont accumulées pendant la semaine et posées à la réunion de suivi.
- Les bugs et les tâches sont suivis dans les *Issues* GitHub, avec l'étiquette du domaine (`iot` , `api` , `pwa`) et du sprint.

---

3. Déjà réalisé le 2 novembre (retiré du planning)

- Planification : répartition du travail et planning des 10 sprints (ce document).

- Contrat MQTT : topics et format des messages (état des places, événements d'anomalie, statut du Pi), **à valider ensemble**.
- Contrat des polygones : format du fichier des places (`places.exemple.json`), **à valider ensemble**.
- Dossier `iot/` structuré : README, `requirements.txt` , `.gitignore` .
- En avance sur les sprints 3 et 4 : test de 3 modèles pré-entraînés ; choix de **YOLO-World** pour pré-annoter et de **YOLO11n** pour le Pi ; choix des datasets **toy-car-detection** et **CARPK**.

---

4. Vue d'ensemble des sprints

Sprint	Dates	Livrable principal	Statut
1	2–6 nov.	User stories, contrats restants, VM en HTTPS, PWA de base, photo prise par le Pi	En cours
2	9–13 nov.	Maquette construite, MQTT sécurisé, UML, outil de dessin des polygones	À faire
3	16–20 nov.	Dataset annoté, connexion JWT, polygones enregistrés	À faire
4	23–27 nov.	Modèle YOLO, plan du parking en temps réel (API et WebSocket)	À faire
5	30 nov.–4 déc.	Occupation calculée, serveur qui reçoit MQTT, tableau de bord exploitant	À faire
6	7–11 déc.	Première chaîne complète caméra → écran, réservation	À faire
7	14–18 déc.	Modèle sur le Pi, paiement simulé, plus court chemin	À faire
8	21–25 déc.	Trajet affiché, verrouillage, anomalies détectées, règles d'alerte	À faire
9	28 déc.–1er janv.	Notifications push, statistiques, évaluation du modèle	À faire
10	4–8 janv.	Déploiement final, documentation, manuel, vidéo, démonstration	À faire

---

5. Détail des sprints

Légende : **Eya = A**, **Amal = B**, **Maissa = C**. « Commun » = les trois ensemble.

Sprint 1 (2–6 nov.) : mis à jour

Objectif : poser les bases techniques : chaque partie a son premier « bonjour » et les contrats sont figés.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	Commun : user stories	Commun : user stories	Commun : user stories
Mer–Jeu	PWA : base de la PWA	IoT : Raspberry Pi et caméra	Serveur : VM cloud et HTTPS
Ven	Commun : contrats restants et validation MQTT/polygones	idem	idem

Lun–Mar, ensemble : écrire les *user stories* du conducteur et de l'exploitant, au format « en tant que… je veux… afin de… », chacune avec ses critères d'acceptation (ex. : « en tant que conducteur, je veux être guidé vers la place libre la plus proche, afin de gagner du temps »). Les ranger par priorité et par sprint dans `docs/user-stories.md` .

Eya (PWA) - Créer `pwa/` : `index.html` , `manifest.json` (nom, icônes, couleurs, `display: standalone`), `service-worker.js` (mise en cache de la coquille de l'application). - Vérifier avec Lighthouse que la PWA est **installable** sur mobile et sur ordinateur. - *Fini quand :* l'application s'installe et s'ouvre hors ligne sur une page d'accueil.

Amal (IoT) - Installer le système sur le Raspberry Pi 4, connecter la caméra, activer le Wi-Fi/SSH. - Écrire `iot/capture.py` (OpenCV ou picamera2) : prend une photo et l'enregistre en local, résolution et nom de fichier configurables. - Prendre une photo de test et la placer dans `iot/photos_test/` (non versionnée). - *Fini quand :* `python iot/capture.py` produit une photo nette sur le Pi.

Maissa (Serveur) - Louer/créer la VM IaaS, ouvrir les ports (22, 443, 8883), créer un utilisateur non root, mettre à jour le système. - Installer un reverse proxy (nginx ou Caddy) avec un certificat **Let's Encrypt** (nom de domaine ou sous-domaine gratuit). - Écrire la procédure pas à pas dans `docs/installation-vm.md`. - *Fini quand :* `https://<domaine>/` répond avec un certificat valide.

Vendredi, ensemble - Ajouter à `docs/contrats.md` : le contrat **API** (routes prévues : auth, places, réservations, événements, statistiques), le contrat **push**, le **plan de la maquette** (dimensions, numéros des places, zones interdites, graphe). - Relire et **valider** les contrats MQTT et polygones déjà rédigés. - Revue croisée des trois PR, tag `sprint-1`.

Démo du vendredi : le Pi prend une photo, la VM répond en HTTPS, la PWA s'ouvre et propose l'installation.

Sprint 2 (9–13 nov.)

Objectif : la maquette physique existe, la communication MQTT est sécurisée, la conception est dessinée.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	Commun : maquette	Commun : maquette	Commun : maquette
Mer–Jeu	Serveur : Mosquitto TLS, WildFly, MongoDB	PWA : calibration, dessin des polygones	IoT : annotation du dataset, lot 1
Ven	Commun : conception UML	idem	idem

Lun–Mar, ensemble : construire la maquette (carton ou bois) : places marquées au ruban adhésif, allées, zone interdite, caméra fixée en hauteur sur son support, éclairage LED stable, voitures miniatures de plusieurs couleurs. Respecter le plan défini au sprint 1. Prendre des photos de référence sous plusieurs angles et éclairages.

Eya (Serveur) - Installer **Mosquitto** avec TLS (certificats, authentification par utilisateur/mot de passe, ACL par topic : le Pi ne publie que sur ses topics). - Installer **WildFly** (Jakarta EE 11) en service systemd, et **MongoDB** avec authentification activée, accessible uniquement en local. - Tester : `mosquitto_pub` / `mosquitto_sub` en TLS depuis un autre poste. - *Fini quand :* un message MQTT circule en TLS du Pi (ou d'un poste) vers la VM, et WildFly se connecte à MongoDB.

Amal (PWA, outil de calibration) - Page `pwa/calibration.html` : afficher une photo de la maquette, dessiner des polygones à la souris/au doigt (clic pour chaque sommet), nommer chaque place, choisir son type (normale, PMR, électrique, réservée) et sa zone. - Exporter au format du **contrat des polygones** (coordonnées normalisées ou en pixels, selon le contrat). - *Fini quand :* un polygone dessiné s'exporte en JSON valide, relu par un script.

Maissa (IoT, annotation) - Pré-annoter les photos de la maquette avec **YOLO-World**, puis corriger à la main dans l'outil choisi (Roboflow, CVAT ou Label Studio), classe « voiture ». - Annoter le **lot 1** (≈ 1/3 des images), avec des images variées (lumière, angle, voitures à cheval, places vides). - *Fini quand :* le lot 1 est exporté au format YOLO dans `iot/dataset/` (non versionné, documenté dans le README).

Vendredi, ensemble : diagrammes UML dans `docs/uml/` : cas d'utilisation (conducteur, exploitant, système), diagramme de classes/modèle de données, diagrammes de séquence (réservation, guidage, détection d'anomalie), diagramme de déploiement.

Démo du vendredi : un message MQTT circule en TLS ; on dessine les polygones sur une photo de la maquette.

Sprint 3 (16–20 nov.)

Objectif : dataset complet, authentification, polygones enregistrés côté serveur.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	IoT : annotation, lot 2	Serveur : schéma MongoDB 1 (places, polygones)	PWA : calibration, enregistrement des polygones
Mer–Jeu	PWA : connexion et redirection selon le rôle	IoT : annotation, lot 3	Serveur : JWT, connexion
Ven	Commun : intégration, pull requests, démo	idem	idem

- **Eya (IoT)** : annoter le **lot 2**.
- **Amal (Serveur)** : collections MongoDB `parkings`, `places` (identifiant, type, polygone, zone), `zones` ; accès aux données via une couche DAO ; données d'exemple (*seed*) importables par script. Dans `api/`.
- **Maissa (PWA)** : l'outil de calibration envoie les polygones à l'API (`POST /parkings/{id}/places`) ; affichage des polygones existants.
- **Eya (PWA)** : page de connexion/inscription ; stockage du jeton JWT ; redirection **conducteur** → **front-office**, **exploitant** → **back-office** ; déconnexion.
- **Amal (IoT)** : annoter le **lot 3**. *Tous : dataset complet, découpé en entraînement / validation / test.*
- **Maissa (Serveur)** : endpoints `POST /auth/register` et `POST /auth/login` ; mots de passe hachés (bcrypt ou PBKDF2) ; JWT signé avec rôle et expiration.

Démo du vendredi : connexion puis redirection selon le rôle ; polygones enregistrés et relus par un script.

Sprint 4 (23–27 nov.)

Objectif : un modèle YOLO entraîné et un plan du parking qui se met à jour en direct.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	Serveur : schéma MongoDB 2 (réservations, événements)	PWA : plan SVG du parking	IoT : entraînement YOLO 1
Mer–Jeu	IoT : entraînement YOLO 2	Serveur : API des places et WebSocket	PWA : plan SVG en temps réel
Ven	Commun : intégration, pull requests, démo	idem	idem

- **Eya (Serveur)** : collections `reservations`, `evenements`, `utilisateurs`, `autorisations` ; index (par place, par date) ; documents d'exemple.
- **Amal (PWA)** : plan du parking en **SVG** généré à partir des polygones : places colorées (libre / occupée / réservée / spéciale), numéros, légende.
- **Maissa (IoT)** : **entraînement YOLO v1** (YOLO11n, dataset toy-car-detection + CARPK + notre dataset) ; suivi des courbes, enregistrement du modèle dans `iot/models/`.
- **Eya (IoT)** : **entraînement YOLO v2** après analyse des erreurs de la v1 (faux positifs, voitures manquées) ; ajustement des hyperparamètres et des images d'entraînement.
- **Amal (Serveur)** : `GET /places` (état de toutes les places) et **WebSocket** `/ws/places` qui diffuse chaque changement.
- **Maissa (PWA)** : le plan SVG s'abonne au WebSocket et se met à jour sans recharger la page.

Démo du vendredi : le modèle détecte les voitures de la maquette sur le jeu de test (**seuil de confiance fixé ensemble**) ; le plan change quand l'API envoie un changement.

Sprint 5 (30 nov.–4 déc.)

Objectif : transformer des détections en états de places et les enregistrer.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	Serveur : réception MQTT, événements d'anomalie	IoT : calcul de l'occupation 1 (recouvrement)	Serveur : réception MQTT, état des places
Mer–Jeu	Serveur : JWT, rôles	PWA : interface exploitant, tableau de bord	IoT : calcul de l'occupation 2 (lissage, seuils)
Ven	Commun : documentation	idem	idem

- **Eya (Serveur)** : l'API s'abonne aux topics d'événements ; stocke les événements d'anomalie dans `evenements`. **Puis** : rôles `CONDUCTEUR` et `EXPLOITANT` dans le JWT ; annotations de sécurité sur les routes ; un conducteur reçoit `403` sur toute route d'exploitation.
- **Amal (IoT)** : `iot/occupation.py` : pour chaque place, calcul du **recouvrement** entre la boîte du véhicule et le polygone (bibliothèque `shapely`) ; une place est occupée au-delà d'un seuil de recouvrement (paramétrable).
- **Maissa (Serveur)** : l'API s'abonne au topic d'état des places, met à jour MongoDB, pousse les changements sur le WebSocket.
- **Amal (PWA)** : tableau de bord exploitant : états des places, nombre libres/occupées, mise à jour en direct.
- **Maissa (IoT)** : **lissage** de l'occupation : un changement n'est validé qu'après **3 images identiques de suite** ; réglage des seuils (confiance, recouvrement).
- **Tous (vendredi)** : documentation du sprint dans `docs/`.

Démo du vendredi : avec des messages MQTT de test, l'état des places change en base et sur le tableau de bord.

Sprint 6 (7–11 déc.)

Objectif : première chaîne complète : caméra → Pi → MQTT → serveur → écran. Réservation.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	IoT : client MQTT 1 (publication de l'état)	Serveur : gestion des utilisateurs	PWA : interface exploitant, configuration et utilisateurs
Mer–Jeu	PWA : interface conducteur, réservation	IoT : client MQTT 2 (file locale, statut du Pi)	Serveur : réservation 1 (création, annulation)
Ven	Commun : intégration, pull requests, démo	idem	idem

- **Eya (IoT)** : `iot/mqtt_client.py` : connexion TLS, publication de l'état des places **uniquement lors des changements** (pas à chaque image), reconnexion automatique.
- **Amal (Serveur)** : API `GET/POST/PUT/DELETE /utilisateurs` réservée à l'exploitant ; gestion des autorisations (PMR, électrique).
- **Maissa (PWA)** : back-office : écran de **configuration** (plan, zones, type de place, durée maximale autorisée) et de gestion des utilisateurs.
- **Eya (PWA)** : front-office : liste/plan des places libres et **réservation** d'une place ; écran « mes réservations ».

- **Amal (IoT)** : **file locale** : en cas de coupure Internet, les événements sont stockés (SQLite ou fichier) et renvoyés à la reconnexion ; publication régulière du **statut du Pi** (en ligne, température, fréquence d'analyse).
- **Maissa (Serveur)** : `POST /reservations` (vérifie que la place est libre, la bloque), `DELETE /reservations/{id}` ; un état « réservée » apparaît sur le plan.

Démo du vendredi : première chaîne complète : on déplace une voiture sur la maquette et l'état change sur le tableau de bord en quelques secondes.

Sprint 7 (14–18 déc.)

Objectif : modèle rapide sur le Pi, paiement simulé, premier algorithme de plus court chemin.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	Serveur : réservation 2 (paiement simulé, expiration)	PWA : interface conducteur, paiement et historique	IoT : export NCNN/TFLite
Mer–Jeu	IoT : optimisation de la vitesse sur le Pi	Serveur : guidage 1 (graphe du parking)	Serveur : guidage 2 (Dijkstra/A*)
Ven	Commun : documentation	idem	idem

- **Eya (Serveur)** : `POST /reservations/{id}/paiement` (simulé, sans transaction réelle) ; **expiration** automatique des réservations non honorées (tâche planifiée) ; notification de fin de durée.
- **Amal (PWA)** : écran de paiement simulé ; **historique** des réservations et des paiements.
- **Maissa (IoT)** : exporter le modèle en **NCNN ou TFLite** et le faire tourner sur le Pi.
- **Eya (IoT)** : optimiser : taille d'image, fréquence d'analyse, seuil ; objectif : **moins d'une seconde par image** sur le Pi ; mesurer et noter les résultats.
- **Amal (Serveur)** : modéliser le parking en **graphe** (entrée, intersections, places ; arêtes pondérées par la longueur) stocké en base d'après le plan de la maquette.
- **Maissa (Serveur)** : implémenter **Dijkstra/A*** : `GET /guidage?depart=entree` renvoie le plus court chemin vers la place libre la plus proche.
- **Tous (vendredi)** : documentation du sprint.

Démo du vendredi : on réserve et on paie (simulé) depuis la PWA ; l'API renvoie le plus court chemin de l'entrée vers une place libre.

Sprint 8 (21–25 déc.)

Objectif : guidage complet, détection des anomalies, règles d'alerte.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	PWA : interface conducteur, trajet	IoT : anomalies 1 (voiture à cheval)	Serveur : guidage 3 (verrouillage, recalcul)
Mer–Jeu	Serveur : règles d'alerte 1 (PMR, électrique, réservée)	Serveur : règles d'alerte 2 (ventouse, événements)	IoT : anomalies 2 (zone interdite)
Ven	Commun : documentation et manuel	idem	idem

- **Eya (PWA)** : le **trajet** est tracé sur le plan SVG (de l'entrée à la place proposée) ; mise à jour si la place change.

- **Amal (IoT)** : détecter une **voiture à cheval** : une même boîte recouvre significativement deux polygones, ou dépasse du marquage ; publier l'événement `a_cheval`.
- **Maissa (Serveur)** : **verrouillage** temporaire de la place proposée (2 à 3 minutes) pour qu'aucun autre conducteur ne la reçoive ; si elle est prise entre-temps, **recalcul** et notification au conducteur.
- **Eya (Serveur)** : règle d'alerte **place spéciale** : une place PMR, électrique ou réservée occupée sans autorisation ni réservation correspondante déclenche une alerte (la caméra ne voit pas les badges : la règle s'appuie sur les autorisations en base).
- **Amal (Serveur)** : règle **ventouse** (durée dépassée ou véhicule présent plusieurs jours) ; enregistrement et classification des événements.
- **Maissa (IoT)** : détecter une **zone interdite** : un véhicule présent dans le polygone d'une allée ou d'une sortie ; publier `zone_interdite`.
- **Tous (vendredi)** : début du **manuel utilisateur** (`docs/`).

Démo du vendredi : une voiture à cheval ou en zone interdite crée un événement et une alerte ; deux conducteurs ne reçoivent pas la même place.

Sprint 9 (28 déc.–1er janv.)

Objectif : notifications push, statistiques, évaluation chiffrée du modèle.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	IoT : évaluation 1 (précision, rappel)	Serveur : notifications push, serveur	PWA : notifications push, application
Mer–Jeu	PWA : graphiques de statistiques	IoT : évaluation 2 (temps d'inférence, latence)	Serveur : statistiques, serveur
Ven	Commun : manuel et vidéo	idem	idem

- **Eya (IoT)** : évaluer le modèle sur le jeu de test : **précision, rappel, mAP**, matrice de confusion ; tableau dans `docs/`.
- **Amal (Serveur)** : **Web Push** côté serveur : génération des clés **VAPID**, enregistrement des abonnements, envoi des notifications (alerte à l'exploitant ; réservation, fin de durée à libérer au conducteur).
- **Maissa (PWA)** : demande de permission, abonnement via le **service worker**, affichage des notifications, clic qui ouvre la bonne page.
- **Eya (PWA)** : graphiques de statistiques : taux d'occupation, heures de pointe, durée moyenne, nombre d'infractions.
- **Amal (IoT)** : évaluer le **temps d'inférence** et la **latence** de bout en bout (changement sur la maquette → affichage).
- **Maissa (Serveur)** : `GET /stats` : agrégations MongoDB (par heure, par jour, par place).
- **Tous (vendredi)** : manuel utilisateur et **scénario de la vidéo**.

Démo du vendredi : une anomalie déclenche une notification dans le navigateur de l'exploitant ; les graphiques s'affichent.

Sprint 10 (4–8 janv.)

Objectif : livrer. Rien de nouveau ne démarre cette semaine, sauf réglages et finitions.

Créneau	A : Eya	B : Amal	C : Maissa
Lun–Mar	Serveur : déploiement final de l'API et HTTPS	PWA : interface exploitant, alertes et finitions	IoT : réglage des seuils sur la maquette
Mer–Jeu	Commun : documentation technique	Commun : manuel utilisateur	Commun : vidéo de démonstration
Ven	Commun : démonstration finale	idem	idem

- **Eya** : déploiement final du `.war` sur WildFly, HTTPS, sauvegarde de MongoDB, procédure d'installation à jour.
- **Amal** : alertes lisibles, messages d'erreur clairs, accessibilité et affichage mobile ; **manuel utilisateur** (`docs/`).
- **Maissa** : réglage définitif des seuils (confiance, recouvrement, lissage) ; **vidéo de démonstration (5 minutes maximum)**.
- **Tous** : README du dépôt complet, **tags `sprint-1` à `sprint-10`**, collection **Postman** de l'API, documentation technique, répétition de la démonstration.

Démo finale : parcours complet du conducteur (inscription, réservation, paiement simulé, guidage, notification) et de l'exploitant (tableau de bord, alerte, statistiques, configuration) sur la maquette.

---

6. Livrables finaux et responsabilité

Livrable	Où	Responsable principal en fin de projet
Code source (API, PWA, passerelle IoT)	GitHub	Tous
Documentation technique et installation	`docs/`	Eya
Manuel utilisateur	`docs/`	Amal
API testable avec Postman	`docs/postman/`	Eya
Vidéo de démonstration (5 min max)	à la racine ou lien dans le README	Maissa
Supports marketing et dossier de presse	`docs/marketing/`	Tous (sprints 9–10)
Tableau d'évaluation du modèle	`docs/`	Eya